



Static Website

What is it really?

- > it is a site that is built with pages of static content or plain HTML, Javascript, or CSS code
 - has pre-built HTML, Javascript, or CSS files directly to users
 - pages are delivered exactly as stored, with no server-side processing
- > the content stays consistent regardless of the user or where the user is
 - it only changes when a developer updates the source code
- > it only offers little interactivity
- > powers many pages you use daily, from documentation and landing pages to portfolios
 - ideal for documentation sites, personal blogs, portfolio websites, and small business sites
- > this architectural choice creates distinct performance, security, and maintenance advantages that developers rely on for critical projects
 - static approach is a practical choice
- > due to the content served is not dependent on a particular user it can serve pre-rendered HTML, Javascript, or CSS without server-side logic, offering fast load times and strong security

It can run on 3 major technologies

- > HTML for structure and content
- > CSS for styling and layout
- > JavaScript for client-side interactivity

Two key characteristics define static websites

- > Content Consistency
 - All users see the same page, regardless of location or device.
- > Pre-rendered Pages
 - Files are built before deployment and are not generated at request time
 - the server simply delivers existing files without processing

Advantages and Disadvantages

Advantages

- > **Fast Performance** - Pre-rendered HTML loads quickly, improving user experience and SEO (search engine optimization)
 - **Instant Page Load Times:** Static websites load instantly. Since pages are pre-built and ready to serve, visitors get content without database queries or server processing delays
 - amplifying this speed (CDN) by distributing files across global edge servers. Your site stays fast even during traffic spikes when dynamic sites slow down.
- > **Enhanced Security** - No server-side processing or databases reduce attack surfaces
 - Naturally secure

- With no databases or server-side processing, they sidestep common vulnerabilities:
 - No SQL injection risks
 - Minimal attack surface
 - No server-side code execution issues
 - Read-only deployment blocks unauthorized changes

> **Low Hosting Costs** - Can be hosted on inexpensive platforms or even free static site hosts

- Without databases or complex back-ends, hosting costs drop dramatically
- The savings multiply through:
 - Simpler development cycles
 - Zero database management
 - Clean, maintainable code
 - Efficient scaling without cost spikes

> **Easy Deployment** - Simple to deploy; often just requires uploading files to a server or CDN (content delivery networks)

> **Great for Simple Content** - Ideal for blogs, portfolios, documentation, and landing pages

> **Modern Tooling Support** - Static site generators (e.g., Hugo, Jekyll) and headless CMS (content management system) options (e.g., Strapi) expand functionality

> **Reliability and Stability** - Static websites stay up

- They have fewer moving parts that can fail, no databases to

crash, and minimal complex code to debug.

- This means:
 - Steady performance under load
 - Maximum uptime
 - Consistent cross-browser behavior
 - Reduced back-end dependencies

Disadvantages

> **Limited Interactivity** - Requires JavaScript or external services for dynamic features (e.g., forms, comments)

> **Manual Updates** - Updating content often means rebuilding and redeploying the entire site

> **No Real-Time Data** - Not suitable for applications needing live data (e.g., stock tickers, dashboards)

> **Scalability Challenges for Large Sites** - Managing hundreds or thousands of pages can become complex

> **User-Specific Content Limitations** - Difficult to serve personalized or authenticated content without extra infrastructure

How Static Websites Deliver Content?

Content Delivery Process

> The delivery flow is efficient and straightforward:

- The browser requests a page
- The server returns the matching HTML file
- The browser loads and renders the content along with CSS and JavaScript

- > There are no runtime database queries or server-side logic
- > Everything is ready to serve at the request time.

Simple Static Sites

- > it's a good option for sites where the content does not change often or when the site is straightforward

Static Site Generators (SSGs)

- > it may supplement more complex use cases, especially useful when you have hardcoded content or content in the markdown files. You reuse a page or templates, and you have content maintained by technical people who can edit content directly in code or are comfortable doing it in markdown files.

Explanation: Static Site Generators (SSGs) are tools that create websites made of pre-built HTML pages instead of generating them on demand. They are especially useful when content is hardcoded or written in Markdown files, such as blogs or documentation. SSGs allow developers to reuse templates and page layouts, making it easy to manage many pages with the same design. They work best when technical users maintain the content, since updates are done by editing code or Markdown files directly, resulting in fast, secure, and simple websites.

- > Popular SSGs include:

- **Next.js** – Leading SSG with static and server rendering
- **Astro** – Emphasizes 'Islands architecture' for selective JavaScript loading

- **Hugo** – Extremely fast; builds thousands of pages in seconds
- **Jekyll** – Simple to use and tightly integrated with GitHub Pages

- > Typical SSG workflow:

- Write content in Markdown, MDX, or via CMS (e.g., Strapi)
- Define templates and layout components
- Build step generates static HTML, CSS, and JS
- Deploy to a static host or CDN

- > This decouples content from presentation while delivering performant, production-ready files.

Modern Hosting Options

- > Specialized platforms handle the build-deploy-deliver flow:

- **Netlify**: Git-based deployment, serverless functions, global CDN.
- **Vercel**: Optimized for Next.js and hybrid sites.
- **GitHub Pages**: Free hosting for repositories.
- **Cloudflare Pages**: Global CDN with static hosting.

- > For projects using headless CMS, various headless CMS hosting options are available. These platforms provide:

- Automatic builds on code push.
- Preview deployments for testing.
- Global CDN distribution.
- SSL certificate management.
- Edge functions for dynamic features.

The Jamstack approach

- > combines static delivery with APIs and client-side JavaScript to create fast, scalable, and interactive experiences

- **Jamstack**: an architecture that expands static sites while preserving their benefits through:
 - JavaScript: Client-side interactivity post-load.
 - APIs: Third-party services and custom functions.
 - Markup: Pre-built HTML from build time.

> This splits the front-end presentation from the back-end data. A Jamstack e-commerce site might:

- Serve static product pages via CDN.
- Load personalized recommendations with JavaScript.
- Process payments through APIs.
- Handle forms with serverless functions.

Static site generator + a headless CMS

> This method is mainly employed where website and content relations are more complex and non-technical editors manage your content

- they don't have to be purely non-technical, but they are uncomfortable pushing changes to the repo every time a content change is made

How to build one?

1. Choose a frontend framework
2. Choose a headless CMS
3. Choose a hosting platform



Dynamic Website

What is it really?

> a website that changes in response to user interactions, offering features such as live inventory displays and customized experiences for each visitor

> it serves content based on the user

> considerations such as language, location, and target group are taken into account, and content is served based on the information relevant to the user

> a set of pages, the content of which changes depending on the visitors' locations, past actions taken on the site, time zones, and more.

> in addition to HTML, CSS, and JavaScript, a dynamic website uses a server-side scripting language like PHP or Python. This enables connection with the database to allow for interactive features and content changes

> Can be used in eCommerce Websites, Forum sites, news websites, and social media sites

> it has a feature of having Custom Recommendations Based on Past Searches it is where they let you track visitors' session information and use it to display content based on their preferences

> it has Dynamic Visuals and Animations that besides the quality content, visuals are crucial for a successful website. High-quality images, dynamic visuals, and animations help

keep visitors engaged, preventing them from leaving your site immediately
> its Format Changes Based on Device
Since around 60% of users access the internet using a phone, having a mobile-friendly website is crucial.

Otherwise, your website may become unusable on a smaller screen, creating an unpleasant user experience
> it also has a feature “Language Based on Country” where Dynamic websites can change their content language automatically based on visitors’ location
> has Social Media Integration where Dynamic websites let you integrate social media platforms to add more functionality

It can run on 5 technologies

> HTML, CSS, Javascript, PHP, or Python

Key characteristics define Dynamic websites

> ability to deliver personalized, real-time content that changes based on user interaction, preferences, or database updates, using server-side scripting and databases for features like user logins, e-commerce, content management, and interactive elements, making them highly engaging and adaptable compared to static sites

Advantages and Disadvantages

Advantages

> **Personalized User Experience** - One of the greatest advantages of dynamic websites is their ability to deliver a personalized experience for users. By monitoring user behavior, preferences,

or location, dynamic websites present tailored content in real-time. This capability is especially beneficial for e-commerce sites, news platforms, and content-driven websites.

> **Automatic Content Updates** - Dynamic websites can automatically update content without requiring manual intervention from a developer. This feature is particularly advantageous for websites that need to display frequently changing information, such as news sites, blogs, or social media networks.

> **Greater Interactivity** - Dynamic websites include a wide range of interactive features that engage users, such as forms, comment sections, live chats, and interactive maps. These elements encourage users to interact with the site, which can increase engagement and time spent on the website.

> **Scalability and Flexibility** - Dynamic websites can easily scale to accommodate increasing amounts of content or users without requiring a complete redesign. The use of content management systems (CMS) allows even non-technical users to manage and update the site, making it flexible and adaptable to changes.

> **Better SEO Opportunities** - Dynamic websites are often more SEO-friendly than static websites. Because they can automatically generate unique metadata (e.g., page titles, descriptions, and URL structures), dynamic sites are better suited to search engines. Additionally, the regular content updates improve the site’s visibility and ranking on search engines.

> **Data-Driven Insights** - Dynamic websites can gather valuable user data, such as site visits, clicks, and interaction rates. This data can help businesses make informed decisions about content strategy, marketing, and user experience improvements

Disadvantages

> **Higher Development and Maintenance Costs** - Dynamic websites are more complex to build and maintain than static websites. The development process involves server-side programming, database integration, and continuous maintenance, which requires a higher level of technical expertise. Over time, there are also ongoing updates, troubleshooting, and security patching to keep the site running smoothly.

> **Expensive Hosting Needs** - Dynamic websites rely on databases and server-side scripting, which require more powerful hosting environments. Hosting a dynamic site is generally more expensive than hosting a static site because it demands more processing power and larger servers capable of supporting real-time content updates.

> **Slower Loading Time** - Dynamic websites are typically slower than static websites because they generate content on the fly by querying databases. If the site is not properly optimized, this can result in slow loading times, negatively affecting user experience and increasing bounce rates.

> **Security Risks** - Because dynamic websites are often connected to databases and incorporate complex features, they are more vulnerable to

cyberattacks, such as SQL injection, cross-site scripting (XSS), and data breaches. Ensuring the security of dynamic websites requires careful attention to detail, robust coding practices, and regular security audits.

> **Needs Constant Content Management**

- Although dynamic websites can automatically update content, they still require substantial management, particularly when dealing with large volumes of content. Content management systems (CMS) need to be properly configured, and ongoing oversight is necessary to ensure content quality, accuracy, and relevance.

> **A Likely Candidate for**

Over-Engineering - As dynamic websites evolve and more features are added, they can become overly complex, which may make troubleshooting more difficult. Over-engineering a site can lead to a confusing user experience, with too many interactive elements or a cluttered design.

How Dynamic Websites Deliver Content?

Client-Side Scripting

> A client-side scripting dynamic website changes its content in response to user actions on the page, such as clicking a button.

> client-side scripted website content is loaded in the visitor's web browser instead of the server. This script determines what user interactions trigger the event and which page content changes.

> When building this type of dynamic website, commonly-used client-side scripting languages include JavaScript and VBScript. As it doesn't use any server-side scripting languages, a dynamic site can be hosted statically

Server-Side Scripting

> Server-side scripted web pages' content is rendered on the host and changes before being displayed. This script instructs the application server on how to assemble the requested page.

> Users' requests and inputs on the website can trigger the script. For instance, when users log in to their eCommerce accounts, the script will instruct the server to show their cart items.

> Server-side scripts are used to build an interactive website. They let users enter data into forms or upload images, which will be stored in a database.

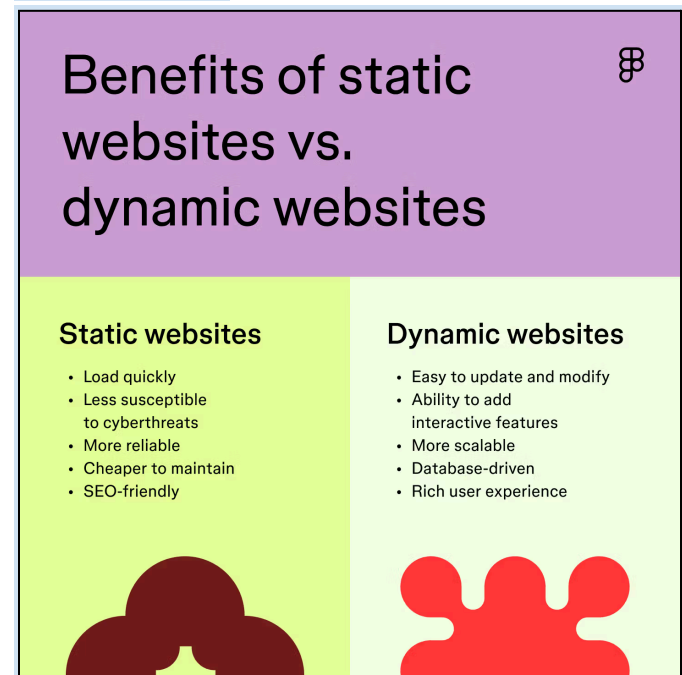
How to build one?

1. Choosing the Right Platform
2. Creating the Dynamic Website
3. Adding Functionality
4. Testing and Launching the Website



Static vs Dynamic

From a website



FEATURE	STATIC WEBSITES	DYNAMIC WEBSITES
Content	Remains fixed	Can change based on user input or real-time data
Development	Typically simpler	More complex, requiring server-side scripting and databases
Performance	Generally faster	Can be slower, especially with complex interactions or large databases
Security	Generally more secure	Can be less secure due to potential vulnerabilities in server-side code and databases
Cost	Often more cost-effective	Can be more expensive to develop and maintain
Use cases	Simple websites, brochures, landing pages	E-commerce stores, social media platforms, news websites

How to choose between static vs. dynamic websites

1. Determine your goal

What's the purpose of your website?
Who is your audience?

2. Consider your content

Think about updates, interactivity,
data collection, and integrations.

3. Factor in budget and time constraints

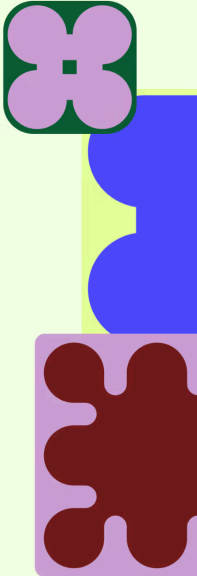
Dynamic websites typically require more
time and money vs. static websites.

4. Evaluate your technical knowledge

Static websites are generally easier to create.

5. Focus on scalability

Looking to grow your site?
Dynamic sites are generally more scalable.



Based from this file

Aspect	Static Website	Dynamic Website
Basic meaning	Built with fixed pages of HTML, CSS, and JavaScript that are delivered exactly as stored	Website content changes based on user interaction, location, preferences, or data

Content behavior	Same content for all users; only changes when a developer updates the source code	Content changes in real time depending on the user, database updates, or interactions
Technologies used	HTML, CSS, JavaScript	HTML, CSS, JavaScript, plus server-side languages like PHP or Python
Server processing	No server-side processing; pre-rendered files are directly served	Uses server-side scripting and databases to generate pages
Interactivity	Limited; relies mostly on JavaScript or external services	Highly interactive; supports logins, forms, comments, live data, and more
Performance	Very fast loading because files are pre-built	Generally slower because pages are generated

	and ready to serve	by querying databases
Security	More secure due to no databases or server-side code	More vulnerable to attacks (e.g., SQL injection, XSS) because of databases and scripts
Hosting cost	Low; can be hosted cheaply or even for free	Higher; requires more powerful hosting and server resources
Content updates	Manual rebuilding and redeployment required	Automatic updates through databases or CMS
Scalability	Can become complex when managing very large sites	Easier to scale for large content and many users
Best used for	Portfolios, blogs, documentation, landing pages, small	E-commerce, social media, news sites, forums,

	business sites	personalized platforms
Examples of features	Pre-built pages, consistent content, fast delivery	Custom recommendations, language by location, user accounts, live data

Similarity	Explanation
Use core web technologies	Both use HTML, CSS, and JavaScript as the foundation
Accessible through browsers	Both are viewed using web browsers
Can be visually designed	Both support layouts, images, animations, and styling
Can be interactive	Both can include interactivity (static uses mostly client-side JS, dynamic uses server-side + client-side)
Can be optimized for users	Both can be built to improve user experience and engagement



Basic HTML programming

> HTML (HyperText Markup Language)

is a markup language, not a programming language. It is the foundational language used to structure content on the web, telling a web browser what to display and how to organize it, rather than executing logic or performing calculations like a programming language (e.g., Python, Java, or JavaScript).

> HTML is a **single markup language, not a collection of languages itself**. It is the foundational language used to structure content on the web.

> HTML works in conjunction with other languages to create modern, interactive web pages. The **three core client-side languages** of the web are:

- HTML
- CSS
- Javascript

> While HTML works with these client-side languages, websites often interact with various **server-side languages** like **Python, PHP, Java, or Ruby** that run on the web server to manage databases and create dynamic content.

> HTML is typically used with Cascading Style Sheets (CSS) for visual presentation and design, and JavaScript for interactivity and dynamic functionality.

What is Basic HTML?

> HTML uses short codes called tags to "mark up" plain text files, giving the content structure and meaning.

HyperText

> Refers to the system of links that connect web pages and websites together, allowing users to navigate the web by clicking from one page to another.

Markup Language

> Refers to the use of tags to annotate text, images, and other content, defining elements such as headings, paragraphs, or links.

How does it work?

> HTML works by creating a set of instructions within a text file that a web browser can read and interpret.

Creation

A developer writes HTML code using a basic text editor and saves the file with a .html or .htm extension.

Request

When a user visits a website, their browser sends a request to a web server for the specified HTML file.

Rendering

The browser receives the HTML document and reads the tags, translating them into a visible, formatted web page for the user to see. The tags themselves are not displayed.

Basic HTML Document

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>

<h1>My First Heading</h1>
<p>My first paragraph.</p>

</body>
</html>
```

> The `<!DOCTYPE html>` declaration defines that this document is an HTML5 document

> The `<html>` element is the root element of an HTML page

> The `<head>` element contains meta information about the HTML page

> The `<title>` element specifies a title for the HTML page (which is shown in the browser's title bar or in the page's tab)

> The `<body>` element defines the document's body, and is a container for all the visible contents, such as headings, paragraphs, images, hyperlinks, tables, lists, etc.

> The `<h1>` element defines a large heading

The `<p>` element defines a paragraph

What is an element?

> An HTML element is defined by a start tag, some content, and an end tag:

`<tagname>` Content goes here `</tagname>`

>The HTML element is everything from the start tag to the end tag:

`<h1>`My First Heading`</h1>`

`<p>`My first paragraph`</p>`

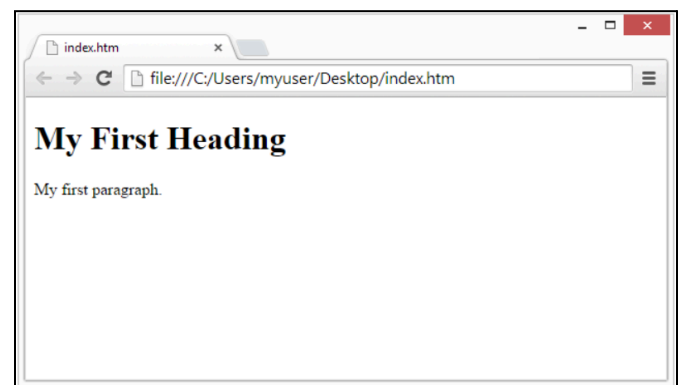
Start tag	Element content	End tag
<code><h1></code>	My first heading	<code></h1></code>
<code><p></code>	My first paragraph	<code></p></code>
<code>
</code>	none	none

Note: Some HTML elements have no content (like the `<br>` element). These elements are called empty elements. Empty elements do not have an end tag

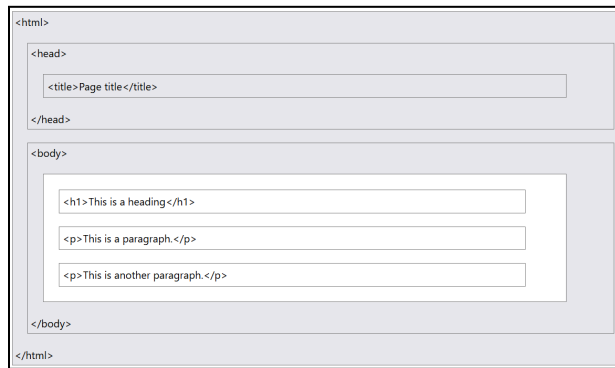
Web browser

> The purpose of a web browser (Chrome, Edge, Firefox, Safari) is to read HTML documents and display them correctly.

> A browser does not display the HTML tags, but uses them to determine how to display the document:



HTML Page structure



Note: The content inside the <body> section will be displayed in a browser. The content inside the <title> element will be shown in the browser's title bar or in the page's tab

Key concepts

HTML is built around elements, tags, and attributes.

Elements: The building blocks of an HTML page. An element consists of an opening tag, the content, and a closing tag

Noteeeeeeeeeeeeeeeeee:

Mga source ko is ummm

- <https://strapi.io/blog/what-is-a-static-website>
- <https://www.verbat.com/blog/advantages-and-disadvantages-of-dynamic-websites/>
- <https://hygraph.com/blog/what-is-a-static-website#what-is-a-dynamic-website>
- [https://www.hostinger.com/ph/tutorials/dynamic-website#Building a Dynamic Web site](https://www.hostinger.com/ph/tutorials/dynamic-website#Building_a_Dynamic_Web_site)
- <https://www.figma.com/resource-library/static-vs-dynamic-website/>
- https://www.w3schools.com/html/html_intro.asp

Chatgpt for the two table ung differences and similarities and google ai para kay html omsim HAHAAHAHAHAAH DI AKO SURE KUNG NASAGOT KO PINAPAHANAP NI SIR PACHECK NLNG

Example:

<p>This is a paragraph.</p>).

Tags: Enclosed in angle brackets

Example:

<body>

> tags tell the browser where an element begins and ends. Some tags, like (for images) or
 (for line breaks), are "empty" and do not require a closing tag.

Attributes: These are used within the opening tag to provide additional information or properties for an element

Example:

the src attribute in

 specifies the image source